

PROVA TÉCNICA SUBITER

Resolução das atividades

Augusto Batista

Implementações em C++/Qt e Flutter

Código-fonte: github.com/Augustob790/Subiter_test

Neste documento apresento as decisões que tomei em cada atividade, os pontos principais do código e a forma de executar os dois projetos.

15 de julho de 2026

1. Inspeções industriais em C++ e Qt

Como organizei a solução

Optei por concentrar os dados comuns em `Inspection`: código, inspetor, data e resultado. Ela é abstrata porque uma inspeção genérica não tem relatório próprio. Quem define esse conteúdo são `ThermographicInspection` e `UltrasoundInspection`.

O método `commonReportData()` evita repetir a montagem do cabeçalho. Cada classe derivada acrescenta somente sua medição: temperatura máxima ou espessura. Assim, se surgir outro tipo de inspeção, basta criar uma nova classe e implementar `generateReport()`.

Visão das classes

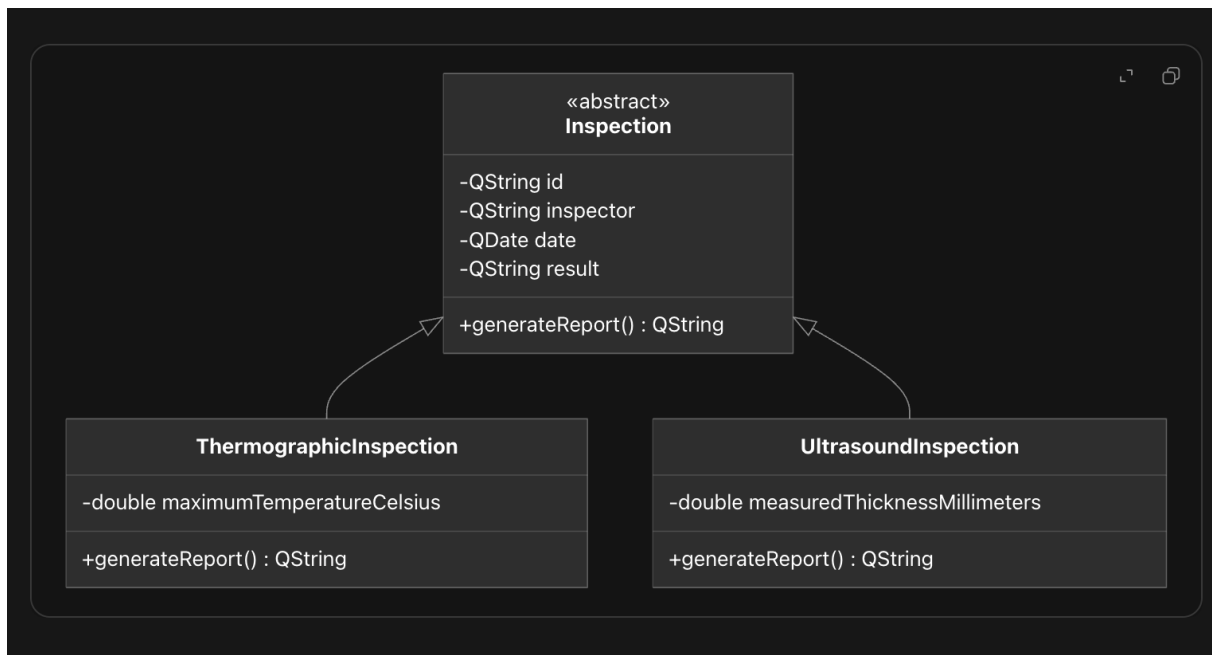


Diagrama de classes usado na modelagem das inspeções.

Uso do polimorfismo

```
std::vector<std::unique_ptr<Inspection>> inspections;
inspections.push_back(
    std::make_unique<ThermographicInspection>(/* dados */));
inspections.push_back(
    std::make_unique<UltrasoundInspection>(/* dados */));

for (const auto &inspection : inspections) {
    qInfo().noquote() << inspection->generateReport();
}
```

Esse trecho é o que demonstra o polimorfismo de fato: o laço conhece somente `Inspection`, mas cada objeto gera o relatório correto. O uso de `unique_ptr` também deixa clara a propriedade dos objetos e evita gerenciamento manual de memória.

Em termos de SOLID, a decisão mais relevante aqui é manter o consumidor dependente da abstração. SRP, OCP e LSP aparecem como consequência dessa modelagem, e não como camadas adicionais sem necessidade.

Código: `subiter_test_c_qt/question1.h` e `subiter_test_c_qt/question1.cpp`. O único `main()` do projeto fica em `subiter_test_c_qt/main.cpp`.

2. Lista de inspeções consumindo uma API REST simulada

O que foi implementado

Como não havia um backend para o exercício, usei assets/mocks/inspections.json como resposta da API. O arquivo mantém um envelope parecido com o de um endpoint real, com statusCode, message e data.inspections. A intenção foi exercitar o mesmo parsing e o mesmo tratamento de falhas que eu usaria com uma chamada HTTP.

```
{
  "statusCode": 200,
  "message": "Inspeções carregadas com sucesso",
  "data": { "inspections": [ ... ] }
}
```

Separação entre tela e lógica

A tela apenas observa o InspectionsViewModel pelo Provider. Ela não abre o JSON, não consulta SQLite e não conhece o formato externo. O ViewModel chama GetInspectionsUseCase, que depende do contrato InspectionsRepository. A implementação concreta fica em infra.

```
InspectionsScreen
-> InspectionsViewModel (ChangeNotifier)
-> GetInspectionsUseCase
-> InspectionsRepository
-> JSON simulado + cache SQLite
```

Estado e erros

Mantive estados explícitos para initial, loading, success, empty e error. Isso deixa o build da tela previsível: cada estado tem uma representação e o botão de nova tentativa apenas dispara o carregamento novamente.

No repositório, primeiro tento a fonte que simula o servidor. Quando a resposta é válida, substituo o cache SQLite. Se a leitura falhar, procuro a última lista salva. Só devolvo erro para a tela quando também não há cache. Prefiro esse fluxo porque ele mantém a listagem útil mesmo em uma falha temporária.

Para trocar o mock por uma API real, seria necessário substituir somente a implementação de InspectionsRemoteDataSource. Tela, ViewModel, caso de uso e domínio permanecem iguais.

Resultado da questão 2



Lista carregada a partir do contrato JSON, com tipo, resultado, responsável, local e medição de cada inspeção.

3. Cadastro de três equipamentos em C++

Escolha da estrutura

O enunciado limita o cadastro a exatamente três equipamentos. Por isso, usei um `std::array` com capacidade fixa em vez de um `vector` que cresceria sem limite. Cada posição é um `std::optional<Equipment>`, pois um espaço ainda não preenchido não deve ser representado por um equipamento falso com ID zero e textos vazios.

```
class EquipmentRegistry final {
public:
    static constexpr std::size_t capacity = 3;
    void add(Equipment equipment);
    void list(std::ostream &output) const;
private:
    std::array<std::optional<Equipment>, capacity> equipments_;
    std::size_t size_ = 0;
};
```

Validação da entrada

O ID só é aceito quando é inteiro e positivo. Em uma entrada inválida, limpo o estado de `std::cin` e descarto o restante da linha antes de pedir o valor novamente. Código, nome e descrição são lidos com `getline`, portanto aceitam espaços, mas não podem ficar vazios.

Também tratei o fim da entrada. Foi esse caso que apareceu quando o Qt Creator executou o programa sem terminal interativo; por isso o projeto mostrou a mensagem “A entrada de dados foi encerrada”.

Como executar

```
cd subiter_test_c_qt
cmake -S . -B build
cmake --build build
./build/SubiterInspections
```

No Qt Creator, habilite Projects > Run > Run Settings > Run in terminal. A questão 3 depende dessa opção para receber os dados digitados.

Código: `subiter_test_c_qt/question3.h` e `subiter_test_c_qt/question3.cpp`.

4. Cadastro de atividade em Flutter

Organização da funcionalidade

A tela pede nome da empresa e local. Mantive Activity como nome da entidade de domínio, seguindo o enunciado, e agrupei as telas no módulo companies, que é como a funcionalidade aparece para o usuário.

Ao tocar em Cadastrar, a página envia os valores para ActivityRegistrationViewModel. O ViewModel não grava no banco diretamente: ele chama RegisterActivity, que remove espaços nas extremidades, valida os dois campos e usa o contrato ActivitiesRepository. A implementação desse contrato persiste o registro na tabela activities do SQLite.

```
ActivityRegistrationPage
-> ActivityRegistrationViewModel
-> RegisterActivity
-> ActivitiesRepository
-> ActivitiesRepositoryImpl
-> SQLite
```

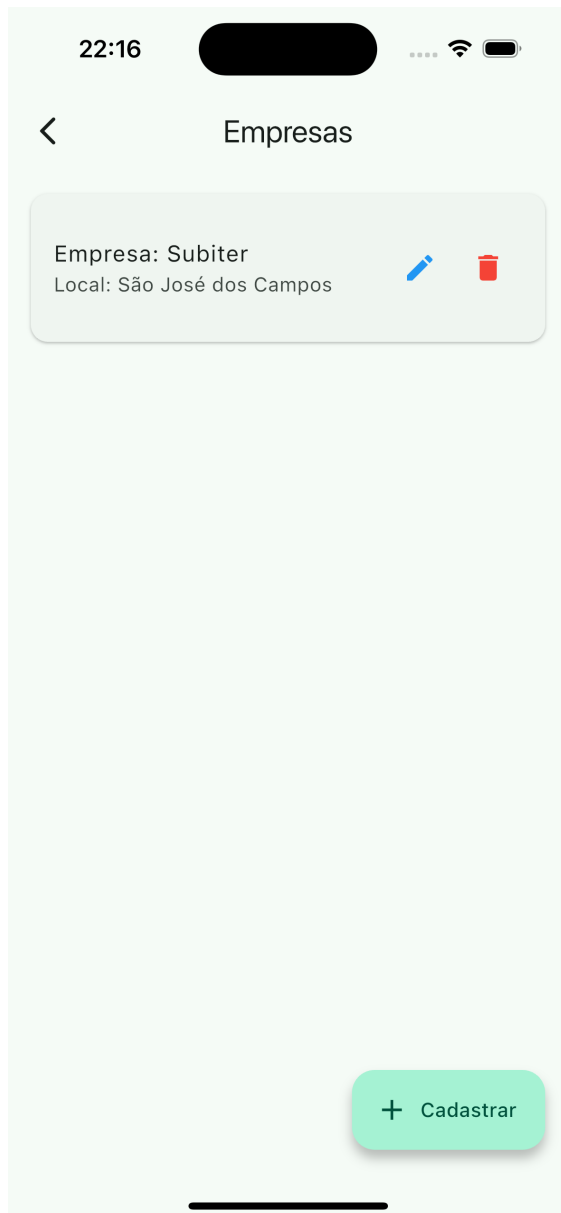
Comportamento da tela

Durante a gravação, o estado passa para saving e novas submissões são bloqueadas. Em caso de sucesso, o objeto salvo fica disponível para montar a confirmação com os dados realmente digitados. Em caso de falha, a tela recebe um estado próprio e continua responsável apenas por mostrar a mensagem adequada.

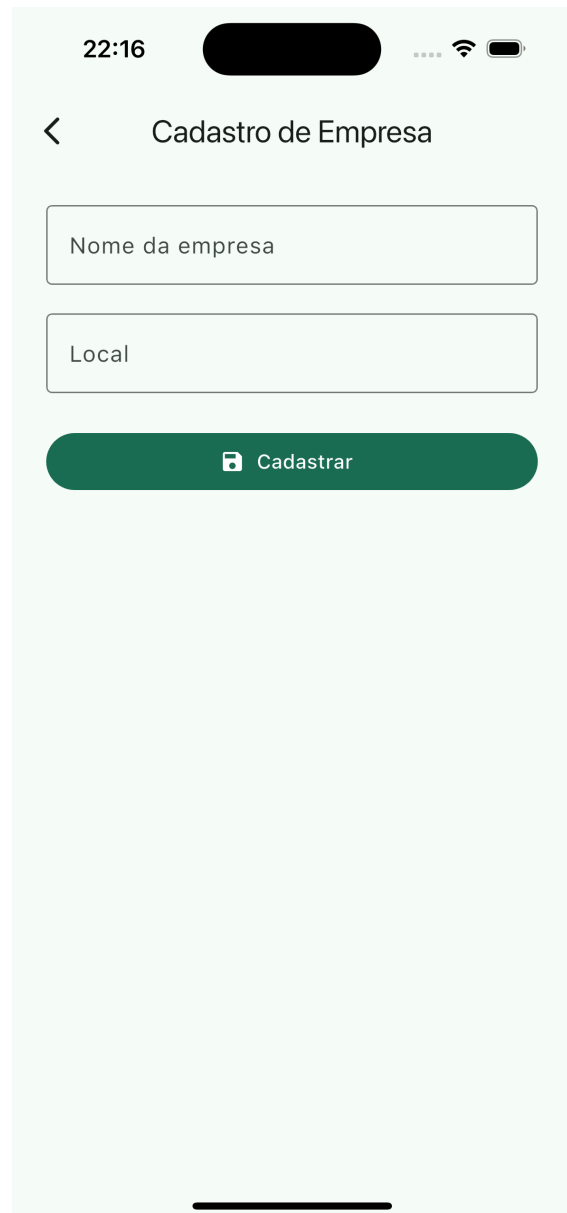
A mesma tela também recebe um item selecionado para edição. A listagem permite atualizar e excluir registros, sempre passando pelas classes do módulo em vez de acessar o banco a partir do widget.

Rotas: /activities para a lista e /activities/register para o formulário. Os textos visíveis usam o catálogo de internacionalização e as dependências são registradas no GetIt.

Resultado da questão 4



Lista com registro persistido.



Formulário de cadastro.

Os projetos são independentes na execução, mas ficam no mesmo repositório: Flutter na raiz e C++/Qt em subiter_test_c_qt/.

Código completo: github.com/Augustob790/Subiter_test